Operating OpenBMC at Hyperscale

What Works, What Breaks, and What Must Change

Hari Ramachandran | Principal Engineer, Microsoft

Thirupathaiah Annapureddy | Partner Firmware Engineering Manager, Microsoft

OCP Global Summit 2026

Session Agenda

A production operating perspective

- **WHAT WORKS**
  - Why OpenBMC delivers value at scale
- **WHAT BREAKS**
  - How flexibility becomes fleet complexity
- **PRACTICAL LESSONS**
  - Patterns that emerge only in production
- **WHAT MUST CHANGE**
  - Where shared standards and OCP alignment can help

What Works: Real Operational Value

OpenBMC is no longer experimental—it is operational at hyperscale.

- **CONTROL**
  - Full-stack ownership with no vendor lock-in on critical management and security paths
- **VELOCITY**
  - Ship firmware updates on the operator timeline—not a proprietary vendor release cycle
- **COLLABORATION**
  - A shared upstream reduces duplicated engineering across the ecosystem
- **AUTOMATION**
  - Standards-based management interfaces, such as Redfish, provides a common foundation for fleet control-plane integration



What Works: Proven at Hyperscale

Production deployment across the breadth of modern datacenter infrastructure

- **MILLIONS**
  - BMCs deployed globally across workload tiers
- **100+ PLATFORMS**
  - Spanning multiple silicon generations
- **500+ SKUS**
  - Compute, storage, networking, and AI infrastructure
- **5+ ODM PARTNERS**
  - Building on a shared open foundation

This scale validates the model—and reveals the next frontier.



What Breaks: Fleet Consistency and Lifecycle

Small implementation differences become systemic at fleet scale.

- **INCONSISTENT SURFACES**
  - Equivalent conditions appear through different management resources, event models and log paths
- **RELIABILITY AT SCALE**
  - Timing, dependency, and protocol edge cases emerge under production conditions
- **LIFECYCLE VARIATION**
  - Update, rollback, and version behavior require platform-specific handling



What Breaks: Flexibility Becomes Lifecycle Cost

HOW DIVERGENCE GROWS

- ODM variants multiply qualification paths
- Configuration deltas compound across building blocks
- Protocol implementations diverge across silicon and generations
- Lab matrices cannot reproduce every fleet combination

WHERE THE COST APPEARS

- Longer qualification and rollout cycles
- Per-platform control-plane adaptations
- Higher incident diagnosis and recovery effort
- Continuous integration maintenance across the lifecycle

The challenge is no longer adoption—it is maintaining consistency across diversity.

Practical Lessons: What Production Exposes

DEPENDENCIES

- A mistyped systemd dependency can silently remove an ordering guarantee
- Per-device retry assumptions can exceed service-level timeouts at fleet scale

PROTOCOL BOUNDARIES

- MCTP discovery and PLDM behavior vary across endpoint implementations
- Interoperability must be tested—not inferred from protocol support

OBSERVABILITY

- Without common event shapes, fleet correlation becomes manual normalization

LIFECYCLE POLICY

- Update, rollback, attestation, and certificate behavior require explicit contracts



What Must Change: Convergence While Preserving Openness

CONSISTENT MANAGEMENT SURFACES

- Common behavior across management APIs, logs, dumps, telemetry, and firmware update
- Baseline expectations that operators can validate consistently

SECURITY LIFECYCLE

- Aligned attestation, certificate, signing, and root-of-trust behavior

CONFORMANCE BY DESIGN

- Cross-vendor integration scenarios for MCTP, PLDM, SPDM, and Redfish implementations.
- Shared tests derived from production failure patterns

OPERATIONAL FEEDBACK LOOP

- • Move recurring fleet lessons into upstream requirements and tests



Where the OCP Community Can Help

1 REDFISH BMC BASELINE

- Define the minimum interoperable BMC management surface implemented consistently across compute, storage, network, rack-level management aggregators, and other platform contexts.

2 PROTOCOL CONFORMANCE

- Share cross-vendor MCTP, PLDM, and SPDM integration tests

3 SECURITY ALIGNMENT

- Coordinate lifecycle expectations across silicon, ODMs, and operators

4 PRODUCTION FEEDBACK

- Convert operational incidents into shared requirements and regression tests



Call to Action

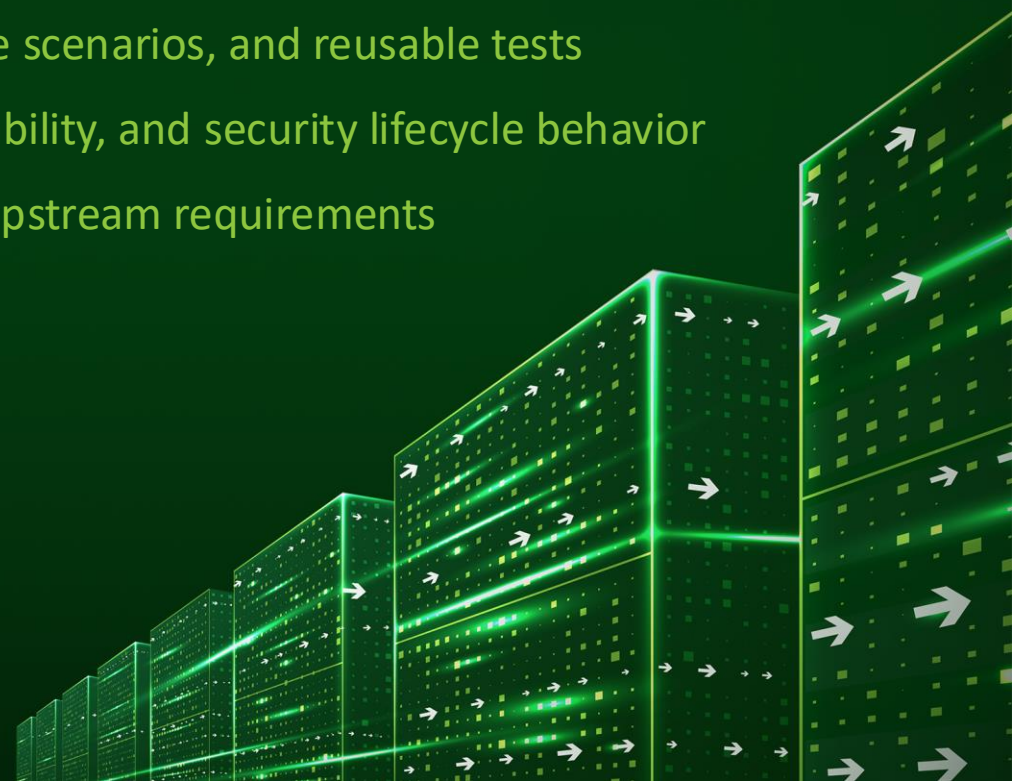
opencompute.org/projects/hardware-management

JOIN — Participate in the OCP Hardware Management project

CONTRIBUTE — Share production patterns, conformance scenarios, and reusable tests

CONVERGE — Define consistent BMC management, interoperability, and security lifecycle behavior

CONNECT — Turn recurring fleet problems into upstream requirements



Open Discussion

- Where does management-surface inconsistency create the greatest fleet cost today?
- Which cross-vendor protocol scenarios should the community test first?
- What minimum BMC management and lifecycle behavior should be consistent across implementations?

Thank You!

